

TP1 — Construire un mini-réseau Linux sans Mininet

Namespaces • veth • bridge Linux • ARP/ICMP • tcpdump • HTTP

Durée conseillée	1 h 45
Niveau	Master — préparation SDN
Plateforme	Ubuntu dans une VM VMware/VirtualBox ; accès sudo
But pédagogique	Construire à la main les mécanismes Linux que Mininet automatisera ensuite.
Topologie finale	h1 — veth — br0 — veth — h2, entièrement à l'intérieur de la VM Ubuntu.

Question fil rouge — Comment faire communiquer deux “machines” virtuelles dans un seul Ubuntu, sans créer deux nouvelles VMs ?

1. Ce que vous allez apprendre

- Comprendre un network namespace : un espace réseau Linux isolé avec ses propres interfaces, adresses et routes.
- Comprendre une paire veth : un câble Ethernet virtuel dont les deux extrémités sont des interfaces Linux.
- Créer un bridge Linux et l'utiliser comme un switch Ethernet logiciel de niveau 2.
- Observer ARP et ICMP avec tcpdump au lieu de se limiter à “ping fonctionne / ne fonctionne pas”.
- Faire circuler un vrai trafic applicatif HTTP entre deux hôtes virtuels.
- Faire le lien avec Mininet : h1/h2 seront des namespaces et le switch sera ensuite généralement Open vSwitch.

À la fin de ce TP, vous devez être capable d'expliquer et de manipuler trois mécanismes fondamentaux de Linux utilisés dans l'émulation réseau :

1. Network namespace

Créer un espace réseau isolé disposant de ses propres interfaces, adresses IP, routes et table de voisinage.

2. Paire veth

Comprendre qu'une paire veth joue le rôle d'un câble Ethernet virtuel à deux extrémités.

3. Bridge Linux

Créer un commutateur Ethernet logiciel de niveau 2 et y raccorder plusieurs hôtes virtuels.

2. Préparation de l'environnement

```
sudo apt update
sudo apt install -y iproute2 iputils-ping tcpdump curl python3
ip -V
ping -V | head -n 1
python3 --version
```

Résultat attendu — Les commandes `ip`, `ping`, `tcpdump`, `curl` et `python3` sont disponibles. Le TP ne modifie pas l'interface réseau principale de la VM.

Attention — N'ajoutez jamais votre interface réelle (`ens33`, `eth0`, `enp0s3`, etc.) au bridge de ce TP. Nous ne travaillons qu'avec des interfaces virtuelles créées pour le laboratoire.

3. État initial : observer le réseau de la VM

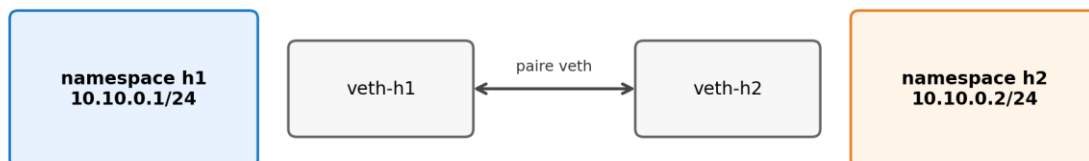
```
ip -br addr
ip route
ip link
```

Questions de compréhension

1. Quelle interface possède l'adresse IP permettant à la VM de communiquer avec le réseau extérieur ?
2. Quelle est la route par défaut ?
3. Pourquoi ne devons-nous pas modifier cette interface pendant ce TP ?

4. Étape A — créer deux hôtes isolés et un câble virtuel

Étape A — un câble Ethernet virtuel entre deux hôtes isolés



Idée : une paire veth se comporte comme les deux extrémités d'un câble.

4.1 Créer les deux network namespaces

```
sudo ip netns add h1
sudo ip netns add h2
ip netns list
```

Résultat attendu — h1 et h2 apparaissent dans la liste. Ils possèdent chacun leur propre pile réseau, mais aucune interface utile n'est encore connectée.

4.2 Créer la paire veth

```
sudo ip link add veth-h1 type veth peer name veth-h2
ip -br link | grep veth
```

À retenir — Une paire veth fonctionne comme un câble : tout paquet entrant d'un côté ressort de l'autre.

4.3 Déplacer une extrémité dans chaque namespace

```
sudo ip link set veth-h1 netns h1
sudo ip link set veth-h2 netns h2
sudo ip -n h1 link
sudo ip -n h2 link
```

4.4 Adresser et activer les interfaces

```
sudo ip -n h1 addr add 10.10.0.1/24 dev veth-h1
sudo ip -n h2 addr add 10.10.0.2/24 dev veth-h2
sudo ip -n h1 link set lo up
sudo ip -n h2 link set lo up
sudo ip -n h1 link set veth-h1 up
sudo ip -n h2 link set veth-h2 up
sudo ip -n h1 -br addr
sudo ip -n h2 -br addr
```

Résultat attendu — h1 possède 10.10.0.1/24 et h2 possède 10.10.0.2/24. Les interfaces sont UP.

4.5 Tester la communication

```
sudo ip netns exec h1 ping -c 3 10.10.0.2
sudo ip netns exec h2 ping -c 3 10.10.0.1
```

Résultat attendu — Les deux pings réussissent. Aucune route par défaut n'est nécessaire : les deux adresses appartiennent au même sous-réseau 10.10.0.0/24.

Questions de compréhension

1. Pourquoi un routeur n'est-il pas nécessaire ici ?
2. Que représente physiquement la paire veth si l'on compare ce montage à un réseau réel ?
3. Un namespace est-il une VM complète ? Quel élément est réellement isolé dans ce TP ?

5. Observer ARP puis ICMP

Ouvrez deux terminaux. Dans le premier, lancez la capture :

```
sudo ip netns exec h1 tcpdump -ni veth-h1 'arp or icmp'
```

Dans le second terminal :

```
sudo ip netns exec h2 ping -c 2 10.10.0.1
```

Résultat attendu — Lors du premier échange, vous devez voir la résolution ARP (“Who has ... ?”) puis les Echo Request / Echo Reply ICMP.

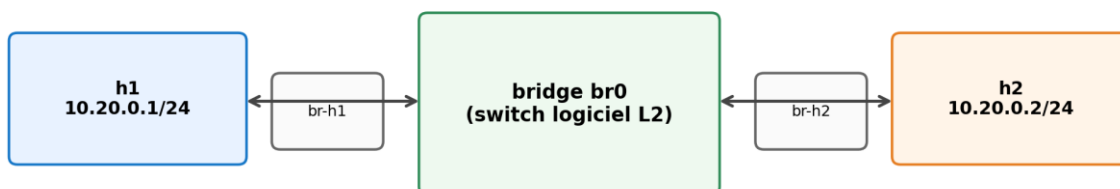
Interprétation — Avant d'envoyer une trame Ethernet à 10.10.0.1, h2 doit connaître l'adresse MAC correspondante. ARP construit ce lien IP <-> MAC.

6. Étape B — remplacer le câble direct par un switch Linux

Nous allons supprimer le montage direct et créer une topologie plus proche de Mininet.

```
sudo ip netns del h1
sudo ip netns del h2
sudo ip link del br0 2>/dev/null || true
```

Étape B — deux hôtes reliés par un switch logiciel Linux



ARP apprend “qui est qui”, puis ICMP circule entre les deux hôtes.

6.1 Créer les namespaces et le bridge

```
sudo ip netns add h1
sudo ip netns add h2
sudo ip link add br0 type bridge
sudo ip link set br0 up
ip -br link show br0
```

6.2 Créer deux “câbles” vers le switch

```
sudo ip link add h1-eth0 type veth peer name br-h1
sudo ip link add h2-eth0 type veth peer name br-h2
sudo ip link set h1-eth0 netns h1
sudo ip link set h2-eth0 netns h2
sudo ip link set br-h1 master br0
sudo ip link set br-h2 master br0
sudo ip link set br-h1 up
sudo ip link set br-h2 up
```

6.3 Adresser les hôtes

```
sudo ip -n h1 addr add 10.20.0.1/24 dev h1-eth0
sudo ip -n h2 addr add 10.20.0.2/24 dev h2-eth0
sudo ip -n h1 link set lo up
sudo ip -n h2 link set lo up
sudo ip -n h1 link set h1-eth0 up
sudo ip -n h2 link set h2-eth0 up
```

6.4 Vérifier le “câblage”

```
ip link show master br0
bridge link
sudo ip -n h1 -br addr
sudo ip -n h2 -br addr
```

Résultat attendu — br-h1 et br-h2 sont membres de br0 ; h1 et h2 sont UP avec leurs adresses 10.20.0.1/24 et 10.20.0.2/24.

6.5 Tester et observer la table MAC

```
sudo ip netns exec h1 ping -c 3 10.20.0.2
bridge fdb show br br0
```

À observer — Après le ping, le bridge a appris des adresses MAC. C’est exactement le comportement attendu d’un switch Ethernet classique.

7. Analyse de trafic sur le bridge

Terminal A :

```
sudo tcpdump -ni br0 'arp or icmp'
```

Terminal B :

```
sudo ip netns exec h1 ping -c 3 10.20.0.2
```

Questions de compréhension

1. Quel protocole apparaît avant ICMP lorsque le cache voisin est vide ?
2. Le bridge travaille-t-il principalement avec des adresses IP ou des adresses MAC ?
3. Quelle différence voyez-vous entre la table ARP/neighbor d'un hôte et la FDB du bridge ?

8. Faire circuler un vrai service HTTP

Dans un premier terminal, démarrer un serveur Web dans h1 :

```
sudo ip netns exec h1 python3 -m http.server 8080 --bind 10.20.0.1
```

Dans un second terminal, depuis h2 :

```
sudo ip netns exec h2 curl http://10.20.0.1:8080
```

Résultat attendu — h2 reçoit une page HTML. Vous venez de faire communiquer une application cliente et un serveur situés dans deux hôtes réseau virtuels différents.

Pourquoi c'est important pour Mininet — Dans Mininet, vous lancerez de la même façon ping, iperf, Python, curl ou d'autres programmes "dans" h1, h2, etc. Les hôtes Mininet exécutent de vrais processus Linux.

9. Mini-défis

1. Ajoutez un troisième namespace h3 avec l'adresse 10.20.0.3/24 et connectez-le à br0 avec une nouvelle paire veth.
2. Vérifiez que h3 peut joindre h1 et h2.
3. Lancez tcpdump sur br0 puis videz le cache neighbor de h3 avec "ip -n h3 neigh flush all". Observez le prochain ping.
4. Expliquez ce qui changerait si h3 recevait l'adresse 10.21.0.3/24 sans qu'aucun routeur ne soit ajouté.

10. Nettoyage obligatoire

```
sudo ip netns del h1 2>/dev/null || true
sudo ip netns del h2 2>/dev/null || true
sudo ip netns del h3 2>/dev/null || true
sudo ip link del br0 2>/dev/null || true
ip netns list
ip -br link | grep -E 'br0|br-h|h[123]-eth' || true
```

Résultat attendu — Aucun namespace h1/h2/h3 ni bridge br0 du TP ne doit rester.

11. Synthèse : le lien direct avec Mininet

Ce que vous avez créé	Rôle	Ce que Mininet fera ensuite
network namespace	Un hôte réseau isolé	h1, h2, h3...
paire veth	Un lien/câble virtuel	liens hôte <-> switch
bridge br0	Commutation Ethernet L2	switch virtuel ; souvent OVS
ip / ping / tcpdump	Configuration et diagnostic	mêmes outils depuis la CLI Mininet

Phrase de conclusion — Mininet ne crée pas un nouveau monde : il automatise massivement des mécanismes Linux que vous venez de construire à la main.

12. Grille de validation rapide

Vérification	Commande / observation	OK
Namespaces créés et inspectés	ip netns list ; ip -n h1 -br addr	[]
Paire veth comprise	ping direct h1 <-> h2	[]
Bridge fonctionnel	ip link show master br0	[]
ARP/ICMP observés	tcpdump -ni br0	[]
Service applicatif testé	python3 -m http.server + curl	[]
Environnement nettoyé	aucun h1/h2/br0 restant	[]